We can then index a couple of movies—*Avatar* (2009) as a blockbuster and *The Matrix Resurrections* (2021) as a flop:

```
PUT blockbusters/_doc/1
{
  "title":"Avatar",
  "blockbuster":true
}

PUT blockbusters/_doc/2
{
  "title":"The Matrix Resurrections",
  "blockbuster":"false"
}
```

In addition to setting the field as JSON's `boolean` type (`true` or `false`), the field also accepts "stringified" Boolean values such as `"true"` and `"false"`, as shown in the second example.

We can use a `term` query (Booleans are classified as structured data) to fetch the results. For example, the following query fetches *Avatar* as a blockbuster:

```
GET blockbusters/_search
{
  "query": {
    "term": {
      "blockbuster": {
        "value": "true"
      }
    }
  }
}
```

We can also provide an empty string for a `false` value: `"blockbuster":""`.

4.5.6 The range data types

The range data types represent lower and upper bounds for a field. For example, if we want to select a group of volunteers for a vaccine trial, we can segregate volunteers based on categories such as age 25–50 or 51–70, demographics such as income level, city dwellers, and so on. Elasticsearch supplies a `range` data type to support search queries on range data. The range is defined by operators such as `lte` (less than or equal to) and `lt` (less than) for upper bounds and `gte` (greater than or equal to) and `gt` (greater than) for lower bounds.

Elasticsearch provides various range data types: `date_range`, `integer_range`, `float_range`, `ip_range`, and more. In the next section, we see the `date_range` type in action.

THE DATE_RANGE TYPE EXAMPLE

The `date_range` date type helps index a range of dates for a field. Then we can use `range` queries to match criteria based on the lower and upper bounds of the dates.